Fast Evaluation of Sequence Pair in Block Placement by Longest Common Subsequence Computation

Xiaoping Tang, Ruiqi Tian, and D. F. Wong, *Member, IEEE*

***Abstract*—Murata *et al.* (1996) introduced an elegant representation of block placement called sequence pair. All block-placement algorithms that are based on sequence pairs use simulated annealing where the generation and evaluation of a large number of sequence pairs is required. Therefore, a fast algorithm is needed to evaluate each generated sequence pair, i.e., to translate the sequence pair to its corresponding block placement. This paper presents a new approach to evaluate a sequence pair based on computing longest common subsequence in a pair of weighted sequences. We present a very simple and efficient $O(n^2)$ algorithm to solve the sequence pair evaluation problem. We also show that using a more sophisticated data structure, the algorithm can be implemented to run in $O(n \log \log n)$ time. Both implementations of our algorithm are significantly faster than the previous $O(n^2)$ graph-based algorithm. For example, we achieve 60× speedup over the previous algorithm when input size $n = 128$. As a result, we can examine a million sequence pairs within one minute for typical input size of placement problems. For all MCNC benchmark block-placement problems, we have obtained the best results ever reported in the literature (including those reported by algorithms based on O tree and B* tree) with significantly less runtime. For example, the best known result for ami49 (36.8 mm^2) was obtained by a B*-tree-based algorithm using 4752 s and we obtained a better result (36.5 mm^2) in 31 s.**

***Index Terms*—Block placement, constraint graph, floorplanning, longest common subsequence, sequence pair.**

I. Introduction

RAPID advances in integrated circuit technology have led to a dramatic increase in the complexity of very large scale integration (VLSI) circuits. According to [13], we will soon have designs in less than 0.1-μm technology with over 100 million transistors. Circuits with such enormous complexity have to be designed hierarchically. Circuit placement within each level of the hierarchy is a complex block-placement problem. A good block-placement solution not only minimizes chip area, but also minimizes interconnect cost which is crucial in determining circuit performance in deep-submicrometer designs. Although block placement is a classical problem with many previous algorithms [12], [14] it remains to be a hard problem. Recently, there were several breakthroughs in block placement—placement representations called sequence pair [7]–[9], BSG [10], O tree [2], [11], and B* tree[1] were invented. In this paper, we shall only focus on the sequence pair representation.

In [7], Murata *et al.* introduced an elegant representation of block placement called sequence pair. All block-placement algorithms that are based on sequence pairs use simulated annealing where the generation and evaluation of a large number of sequence pairs is required. Therefore, a fast algorithm is needed to evaluate each generated sequence pair, i.e., to translate the sequence pair to its corresponding block placement. In [7], an $O(n^2)$ algorithm based on constructing a pair of horizontal and vertical constraint graphs and computing longest paths in both constraint graphs was used for sequence pair evaluation. In [15], Takahashi improved the speed for sequence-pair evaluation by presenting an $O(n \log n)$ algorithm. His algorithm, as presented in [15], determines the width and height of the block placement, but not the positions of the individual blocks. However, the algorithm in [15] can be easily modified to output the positions of all blocks. (Note that it is very important to have the positions of the blocks since they are needed for computing the interconnect cost.)

In this paper, we present a new approach to evaluate a sequence pair based on computing longest common subsequence (LCS) in a pair of weighted sequences. We present a very simple and efficient $O(n^2)$ algorithm to solve the sequence pair evaluation problem. We also show that using a more sophisticated data structure, the algorithm can be implemented to run in $O(n \log \log n)$ time. Both implementations of our algorithm are significantly faster than the previous $O(n^2)$ graph-based algorithm in [7]. For example, we achieved 60× speedup over the original algorithm when the number of blocks is 128. As a result, we can evaluate a million sequence pairs within one minute for typical input size of placement problems. For all MCNC benchmark block-placement problems, we have obtained the best results ever reported in the literature (including those reported by algorithms based on O tree and B* tree) with significantly less runtime. For example, the best known result for ami49 (36.8 mm^2) was obtained by a B*-tree-based algorithm using 4752 s and we obtained a better result (36.5 mm^2) in 31 s.

In the sections that follow, the concept of block placement by sequence pair is reviewed in Section II, LCS is introduced and its relation to sequence pair structure described in Section III, the LCS algorithm as well as its efficiency analysis is presented in Section IV, some experimental results are presented in Section V, and finally, Section VI concludes the paper.

Manuscript received February 7, 2000; revised May 6, 2001. This work was supported in part by the National Science Foundation under Grant CCR-9912390. This paper was recommended by Associate Editor M. Pedram.

X. Tang and D. F. Wong are with the Department of Computer Sciences, University of Texas at Austin, Austin, TX 78712 USA (e-mail: tang@cs.utexas.edu; wong@cs.utexas.edu).

R. Tian is with the Department of Computer Sciences, University of Texas at Austin, Austin, TX 78712 USA and also with Motorola Inc., Austin, TX 78721 USA (e-mail: ruiqi@cs.utexas.edu).

Publisher Item Identifier S 0278-0070(01)10641-X.

0278–0070/01$10.00 © 2001 IEEE

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

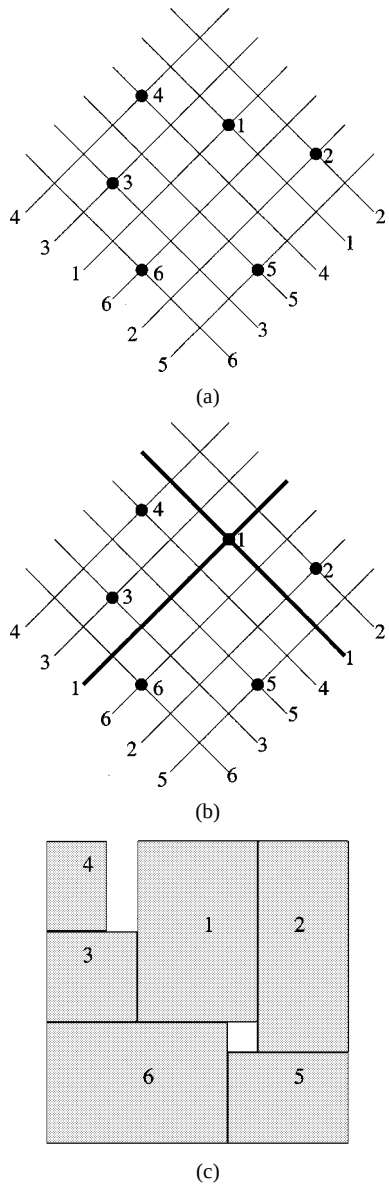


Fig. 1. (a) Oblique grid for sequence pair ($\langle 4\ 3\ 1\ 6\ 2\ 5\rangle$, $\langle 6\ 3\ 5\ 4\ 1\ 2\rangle$), (b) four cones of block 1, and (c) corresponding packing. Dimensions for the six blocks are: 1 (4×6), 2 (3×7), 3 (3×3), 4 (2×3), 5 (4×3), and 6 (6×4).

II. Block Placement by Sequence Pair

A sequence pair is a pair of sequences of n elements representing a list of n blocks. The sequence pair structure is actually a meta grid. Given a sequence pair (X, Y), one can construct a 45 degree oblique grid as shown in Fig. 1(a). For every block, the plane is divided by the two crossing slope lines into four cones as shown in Fig. 1(b). Block 2 is in the right cone of block 1, then it is to the right of block 1 [see Fig. 1(c)]. In general, the sequence pair imposes the relationship between each pair of blocks as follows:

$$(\langle \cdots b_i \cdots b_j \cdots \rangle, \langle \cdots b_i \cdots b_j \cdots \rangle) \Rightarrow b_i \text{ is to the left of } b_j \tag{1}$$

$$(\langle \cdots b_j \cdots b_i \cdots \rangle, \langle \cdots b_i \cdots b_j \cdots \rangle) \Rightarrow b_i \text{ is below } b_j. \tag{2}$$

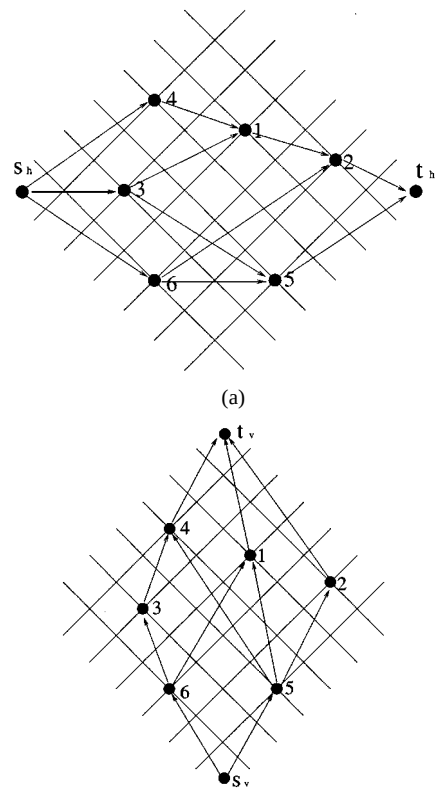


Fig. 2. (a) Horizontal constraint graph $G_h(V, E)$ and (b) vertical constraint graph $G_v(V, E)$ for sequence pair $(X, Y) = (\langle 4\ 3\ 1\ 6\ 2\ 5\rangle, \langle 6\ 3\ 5\ 4\ 1\ 2\rangle)$. (For simplicity, transitive edges are omitted.)

Consequently, given a sequence pair (X, Y), the horizontal relationship among blocks follows a horizontal constraint graph $G_h(V, E)$, which can be constructed as follows:

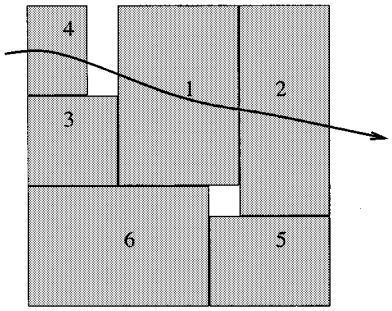
1) $V = \{s_h\} \cup \{t_h\} \cup \{v_i | i = 1, \ldots, n\}$, where v_i corresponds to a block, s_h is the source node representing left boundary and t_h is the sink node representing right boundary;
2) $E = \{(s_h, v_i) | i = 1, \ldots, n\} \cup \{(v_i, t_h) | i = 1, \ldots, n\} \cup \{(v_i, v_j) |$ block i is to the left of block $j\}$;
3) vertex weight equals the width of block i for vertex v_i, but zero for s_h and t_h.

The vertical constraint graph $G_v(V, E)$ can be similarly constructed. As for the example shown in Fig. 1, the corresponding constraint graphs $G_h(V, E)$ and $G_v(V, E)$ are shown in Fig. 2.

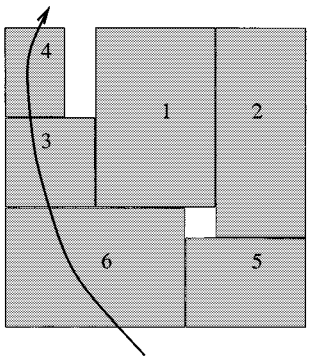
Both $G_h(V, E)$ and $G_v(V, E)$ are vertex-weighted directed acyclic graphs, so a longest path algorithm can be applied to determine the x and y coordinates of each block. The coordinates of a block are the coordinates of the lower left corner of the block.

The construction of constraint graphs G_h and G_v takes $\Theta(n^2)$ time. The longest path computation can be done in $O(n + m)$ time, where $n = \#\text{vertices}$ and $m = \#\text{edges}$ in the graph. The overall time for translating a sequence pair to a placement is then $\Theta(n^2)$. On the other hand, as shown in sections that follow, instead of constructing the graphs G_h and G_v followed by longest path searches, computing the LCS for the weighted sequence pair can efficiently obtain the same optimal placement under the constraints.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

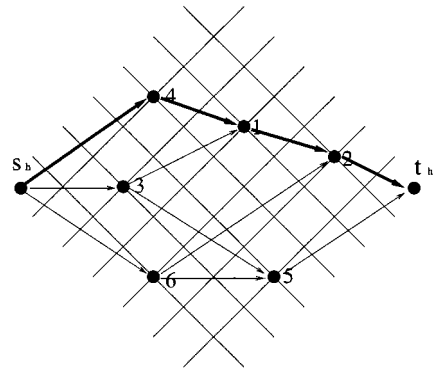


(a)

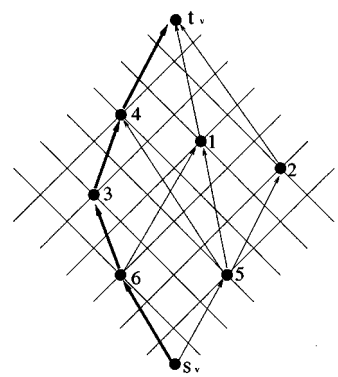


(b)

Fig. 3. For the packing of sequence pair $(X, Y) = (\langle 4\,3\,1\,6\,2\,5\rangle, \langle 6\,3\,5\,4\,1\,2\rangle)$. (a) Common subsequence $\langle 4\,1\,2\rangle$ of (X, Y) shows horizontal constraint. (b): Common subsequence $\langle 6\,3\,4\rangle$ of $(X^R, Y) = (\langle 5\,2\,6\,1\,3\,4\rangle, \langle 6\,3\,5\,4\,1\,2\rangle)$ shows vertical constraint.



(a)



(b)

Fig. 4. (a) In horizontal constraint graph $G_h(V, E)$, a path $s_h \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow t_h$ corresponds to $\langle 4\,1\,2\rangle$, a common subsequence of $(X, Y) = (\langle 4\,3\,1\,6\,2\,5\rangle, \langle 6\,3\,5\,4\,1\,2\rangle)$. (b) In vertical constraint graph $G_v(V, E)$, a path $s_v \rightarrow 6 \rightarrow 3 \rightarrow 4 \rightarrow t_v$ corresponds to $\langle 6\,3\,4\rangle$, a common subsequence of $(X^R, Y) = (\langle 5\,2\,6\,1\,3\,4\rangle, \langle 6\,3\,5\,4\,1\,2\rangle)$.

III. Longest Common Subsequence for Weighted Sequence Pair

In this section, we first introduce the definitions of weighted sequence and LCS, and then show that sequence pair evaluation is equivalent to LCS computation.

Definition 1: A weighted sequence is a sequence whose elements are in a given set S, while every element $s_i \in S$ has a weight. Let $w(s_i)$ denote the weight of s_i.

Note that in the previous sequence definition, elements in S have unit weight, i.e., $w(s_i) = 1$. In this paper, we only consider the situation $\forall s_i \in S$, $w(s_i) \geq 0$.

Definition 2: Given two weighted sequences X and Y, a sequence Z is a common subsequence of X and Y if Z is a subsequence of both X and Y. For example, $\langle 1\,2\rangle$ is the common subsequence of $\langle 1\,5\,2\rangle$ and $\langle 2\,4\,1\,2\,5\rangle$.

Note that element repetition and sequences with unequal length are allowed in the above definition. The sequence in a sequence pair is a permutation of n elements, where no repetition exists and the lengths of the two sequences are equal.

Definition 3: The length of a common subsequence $Z = \langle z_1, \ldots, z_2, \ldots, z_n\rangle$ is $\sum_{i=1}^{n} w(z_i)$.

In other words, the LCS for a sequence pair is the common subsequence of the two sequences with maximum length. In the following, let LCS denote longest common subsequence and $\mathrm{lcs}(X, Y)$ denote the length of the LCS of X and Y.

Given the block set $B = \{1, \ldots, n\}$ and sequence pair (X, Y), a horizontal path in the packing is a common subsequence of (X, Y) as shown in Fig. 3(a) and a path from s_h in horizontal constraint graph corresponds to a common subsequence of (X, Y) as shown in Fig. 4(a). Let X^R denote the reverse of X. Then, a vertical path in the packing is a common subsequence of (X^R, Y) as shown in Fig. 3(b) and a path from s_v in vertical constraint graph corresponds to a common subsequence of (X^R, Y) as shown in Fig. 4(b).

LCS can also determine the position of individual block. Suppose there is a block b in the sequence pair (X, Y). Let $(X, Y) = (X_1 b X_2, Y_1 b Y_2)$. Then, $(X^R, Y) = (X_2^R b X_1^R, Y_1 b Y_2)$. We can see a path from s_h to b in the horizontal constraint graph corresponds to a common subsequence of (X_1, Y_1) [see Fig. 4(a)]. Similarly, a path from s_v to b in the vertical constraint graph corresponds to a common subsequence of (X_2^R, Y_1) [see Fig. 4(b)]. Note that the coordinates of a block are the coordinates of the lower left corner of the block. Thus, if $w(i)$ = width of block i, $\mathrm{lcs}(X_1, Y_1)$ is the x coordinate of block b.

Lemma 1: $\forall b \in B$, if sequence pair $(X, Y) = (X_1 b X_2, Y_1 b Y_2)$, then $\mathrm{lcs}(X_1, Y_1)$ is the x coordinate of block b, where B is block set and $w(i)$ = width of block i. $\mathrm{lcs}(X, Y)$ is the width of the block placement.

Proof: As presented in [7], the longest path from vertex s_h to the vertex v_b representing b in graph $G_h(V, E)$ gives the x coordinate of block b. Therefore, it suffices only to prove that the length of the longest path from s_h to v_b is equal to $\mathrm{lcs}(X_1, Y_1)$. One observation is that the source vertex s_h of weight 0 and the sink vertex t_h of weight 0 can be safely omitted due to the fact that s_h can always be added to the head of both sequences and t_h

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

to the tail without changing the length of common subsequence for any block.

By the definition of the constraint graph in the previous section, a path to v_b in $G_h(V, E)$ corresponds to a common subsequence of (X_1, Y_1) and vice versa. In addition, since the vertex weight in $G_h(V, E)$ for a block is equal to the weight of the corresponding element in the sequence pair, the length of the path to v_b in $G_h(V, E)$ is equal to the length of the corresponding common subsequence of (X_1, Y_1). Thus, the length of the longest path to v_b is equal to the length of the LCS of (X_1, Y_1), i.e., $\mathrm{lcs}(X_1, Y_1)$. Since the path from s_h to t_h corresponds to a common subsequence of (X, Y), the length of the longest path from s_h to t_h is $\mathrm{lcs}(X, Y)$. □

From the fact that

$$(X, Y) = (\langle \cdots b_j \cdots b_i \cdots \rangle, \langle \cdots b_i \cdots b_j \cdots \rangle) \Rightarrow b_i \text{is below} b_j$$

is equivalent to

$$(X^R, Y) = (\langle \cdots b_i \cdots b_j \cdots \rangle, \langle \cdots b_i \cdots b_j \cdots \rangle) \Rightarrow b_i \text{is below} b_j$$

we can similarly conclude the following lemma.

Lemma 2: $\forall b \in$ the block set B, if sequence pair $(X, Y) = (X_1 b X_2, Y_1 b Y_2)$, then $(X^R, Y) = (X_2^R b X_1^R, Y_1 b Y_2)$ and $\mathrm{lcs}(X_2^R, Y_1)$ is the y coordinate of block b, where $w(i)$ is the height of block i and $\mathrm{lcs}(X^R, Y)$ is the height of the block placement.

Based on the Lemmas 1 and 2, we have the following theorem.

Theorem 1: LCS computation can be applied to determine the x and y coordinates of each block and the width and height of the block placement for a given sequence pair.

IV. A Fast LCS Algorithm

A fast algorithm for computing LCS was introduced in [3]. However, it only handles sequences with unit weight. The representation they used is tightly based on unit weight and can not be adapted to the presence of weights. In the rest of this section, we first propose a very simple and efficient $O(n^2)$ algorithm to compute LCS for a given sequence pair, where n is the number of elements and the weights are not restricted to be one or integers. Then, we show that using more sophisticated data structure, the algorithm can be improved to run in $O(n \log \log n)$ time.

In the section, we only focus on how to compute the x coordinates of blocks and the total width of the placement, since computing y coordinates and the height is similar to deal with the sequences (X^R, Y) with weight: $w(b)$ equals the height of block b.

A. Algorithm 1

Assume the blocks are named $1, 2, \ldots, n$ and the input sequence pair is (X, Y). Both X and Y are then permutations of $\{1, 2, \ldots, n\}$. The weight $w(b)$ is equal to the width of block b. Block position array $P[b]$, $b = 1, 2, \ldots, n$ is used to record the x coordinate of block b. For each index i in X, we need a list of corresponding j indexes such that $X[i] = Y[j]$. The array $\mathrm{match}[i]$, $i = 1, 2, \ldots, n$ is constructed for this purpose, i.e.,

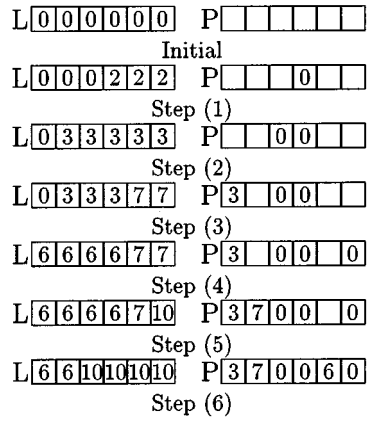


Fig. 5. Example of Algorithm 1 to compute the LCS of sequence pair (⟨4 3 1 6 2 5⟩, ⟨6 3 5 4 1 2⟩). Dimensions of the six blocks are shown in Fig. 1. Here, $w(i)$ is the width of block i, L is the candidate of LCS, and P is the position of blocks.

$\mathrm{match}[i] = j$ iff $X[i] = Y[j]$. The length array $L[1 \cdots n]$ is used to record the length of candidates of the LCS. The algorithm is as follows.

```
Algorithm 1: EVAL-SP1(X, Y)
1   Initialize_Match_Array match;
2   Initialize_Length_Array L with 0;
3   FOR i = 1 TO n DO
4     b = X[i];
5     FOR p on match[i] DO
6         P[b] = L[p];
7         t = P[b] + w(b);
8         FOR j = p TO n DO
9          IF (t > L[j])
10             THEN L[j] = t;
11             ELSE BREAK;
12   RETURN L[n];
```

Since a sequence pair consists of two permutation sequences, each $\mathrm{match}[i]$ contains exactly one entry. As an example, Fig. 5 shows the steps algorithm 1 took to compute the LCS of sequence pair (⟨4 3 1 6 2 5⟩, ⟨6 3 5 4 1 2⟩) in determining the x coordinate of each block. The weights of blocks are their widths as shown in Fig. 1. When the algorithm ends, the array $P[1 \cdots 6]$ records the blocks' x coordinates.

Theorem 2: Algorithm 1 correctly returns $\mathrm{lcs}(X, Y)$ and the position of each block is correctly recorded in $P[1 \cdots n]$ for a sequence pair (X, Y).

Proof: We use an *induction* on i to prove that the positions of the blocks in subsequence $X[1 \cdots i]$ are correctly recorded.

Initially, all the $L[j] (j = 1, \ldots, n)$ is zero. When $i = 1$, the position of block b $(= X[1])$ is zero. Assume that all the positions of the blocks in $X[1 \cdots k]$ are correctly recorded. When $i = k + 1$, we let b_1 denote the block $X[k + 1]$ and p_1 denote the block's index in the sequence Y, i.e.

$$\mathrm{match}[k + 1] = p_1.$$

Therefore, according to *Lemma 1*, the position of block b_1 is $\mathrm{lcs}(X[1 \cdots k], Y[1 \cdots (p_1 - 1)])$. Assume the last element of

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

the LCS is b_2. Let u denote the index of b_2 in X and v denote the index of b_2 in Y. Thus, $u \le k$ and $v \le p_1 - 1$. We get

$$\begin{aligned}&\mathrm{lcs}(X[1\cdots k], Y[1\cdots(p_1-1)])\\ &\qquad = \mathrm{lcs}(X[1\cdots(u-1)], Y[1\cdots(v-1)]) + w(b_2).\end{aligned}$$

Since all the positions of the blocks in $X[1\cdots k]$ are correctly recorded, according to the induction assumption, we get

$$P[b_2] = \mathrm{lcs}(X[1\cdots(u-1)], Y[1\cdots(v-1)]).$$

Then, $\mathrm{lcs}(X[1\cdots k], Y[1\cdots(p_1-1)]) = P[b_2] + w(b_2)$.

The code from line 7 to line 11 guarantees that $P[b_2]+w(b_2)$ is recorded in $L[p_1]$.

When the algorithm ends, $L[n]$ records the maximal value of the position of block b plus its weight ($b = 1, \ldots, n$), which is $\mathrm{lcs}(X, Y)$. □

For a sequence pair, initializing the match array can be done in $O(n)$ time by scanning the two sequences. The total size of match is $O(n)$. The runtime of Algorithm 1 is $O(n^2)$ in worst case,[1] and the space requirement is $O(n)$. In spite of its $O(n^2)$ runtime, the code is very tight, so the hidden constant factor in its runtime is very small.

B. Algorithm 2

We can use a *priority queue* to manage the array $L[1\cdots n]$ in algorithm 1. Johnson [4] introduced an efficient implementation of priority queue over the domain $\{1, \ldots, n\}$. By using the priority queue, we can evaluate a sequence pair in $O(n \log\log n)$ runtime. Every element in the priority queue is associated with two keys: (index, length) and both keys must be kept in increasing (nondecreasing) order. We use *index* as the primary key and discard the element whose *length* is not increasing since it makes no contribution to subsequent LCS computation. In some pass of iteration, the discarding may take $\Omega(n)$ time. However, we will prove the total time charged for discarding elements can be amortized. The algorithm is shown below.

```
Algorithm 2: EVAL-SP2(X, Y)
1    Initialize_Match_Array match;
2    Initialize_Priority_Queue PQueue with
  (0,0);
3    FOR i = 1 TO n DO
4      b = X[i];
5      FOR p on match[i] DO
6        insert (p, 0) to PQueue according
           to 1st key: p;
7        P[b] = predecessor(p).length;
8        t = P[b] + w(b);
9        replace (p, 0) with (p, t);
10       discard the successors of p from
           PQueue whose length ≤ t;
11   RETURN element(index_max).length;
```

The priority queue is implemented as a complete binary tree with $n+1$ leaves on the lowest level. The leaves of the binary tree on the lowest level are numbered consecutively from the left so that they correspond to the index domain $\{0, 1, \ldots, n\}$, where the elements in the priority queue are stored. The elements in the priority queue are kept in a doubly linked list sorted on indexes (refer to Fig. 6). It was shown that insertion and deletion of an element on such a priority queue take $O(\log\log n)$ time. With the help of the doubly linked list, the operations of finding *neighbor* (*predecessor* and *successor*) and retrieving *MIN* and *MAX* take constant time.

As an example, Fig. 6 shows the steps Algorithm 2 took to compute the LCS of sequence pair($\langle 4\ 3\ 1\ 6\ 2\ 5\rangle$,$\langle 6\ 3\ 5\ 4\ 1\ 2\rangle$) in determining the x coordinate of each block. The weights of blocks are their widths as shown in Fig. 1. As in Algorithm 1, the array $P[1\cdots 6]$ is used to record the x coordinate of each block.

Theorem 3: Given a sequence pair (X, Y), algorithm 2 reports the $\mathrm{lcs}(X, Y)$ and records the position of each block in $O(n \log\log n)$ runtime with $O(n)$ space requirement.

Proof: As described before, the x coordinate of block b is $\mathrm{lcs}(X[1\cdots(i-1)], Y[1\cdots(j-1)])$, where $X[i] = Y[j] = b$. The priority queue is kept in increasing order on index. The element in the queue with greater index but less length is useless to subsequent LCS computation. In line 10, the useless elements are deleted. Therefore, $\mathrm{lcs}(X[1\cdots i], Y[1\cdots j])$, where $X[i] = Y[j]$, is indicated by the length t of the element (j, t) in the priority queue. Similar to Algorithm 1, induction on i would prove the correctness of recording the position of each block. The algorithm returns $\mathrm{element}(\mathrm{index}_{\max}).\mathrm{length}$, which is exactly $\mathrm{lcs}(X, Y)$.

Now we use amortized analysis to prove that its runtime is $O(n \log\log n)$. The initializations in lines 1 and 2 take $O(n)$ time. In line 3, the iteration has n passes. For a sequence pair, each $\mathrm{match}[i]$ contains one entry. Lines 4, 7, 8, and 9 take constant time each. As described before, the insertion in line 6 takes $O(\log\log n)$ time. As maintained, the queue is kept in increasing order on both index and length before the insertion of the new element. During the discarding in line 10 after the new element is inserted, we only need to check its successors. We keep checking until an element with greater length than t is found. As shown previously, discarding one element from PQueue takes $O(\log\log n)$ time. Suppose there are d_k elements discarded during the kth pass of the iteration. Thus, the total runtime is

$$O(n \log\log n) + \sum_{k=1}^{n} d_k \cdot O(\log\log n).$$

Since there are at most n elements discarded, $\sum_{k=1}^{n} d_k \le n$. Therefore, the total runtime is $O(n \log\log n)$.

The complete binary tree can be represented as $H = \{1, \ldots, 2^h + n\}$, where n is the size of the index domain and h is the height of the tree $h = \lceil \log(n+1) \rceil$. Thus, $n + 1 \le 2^h < 2(n+1)$ and then the size of the tree is $O(n)$. Clearly, the other space requirement is $O(n)$. □

C. Remark

Note that our algorithms can solve the general LCS problem which also has applications in other VLSI problems (e.g., topo-

[1]The actual complexity depends on the blocks' dimensions. Our experiments show that the complexity is $O(n^{1.48})$ on average.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

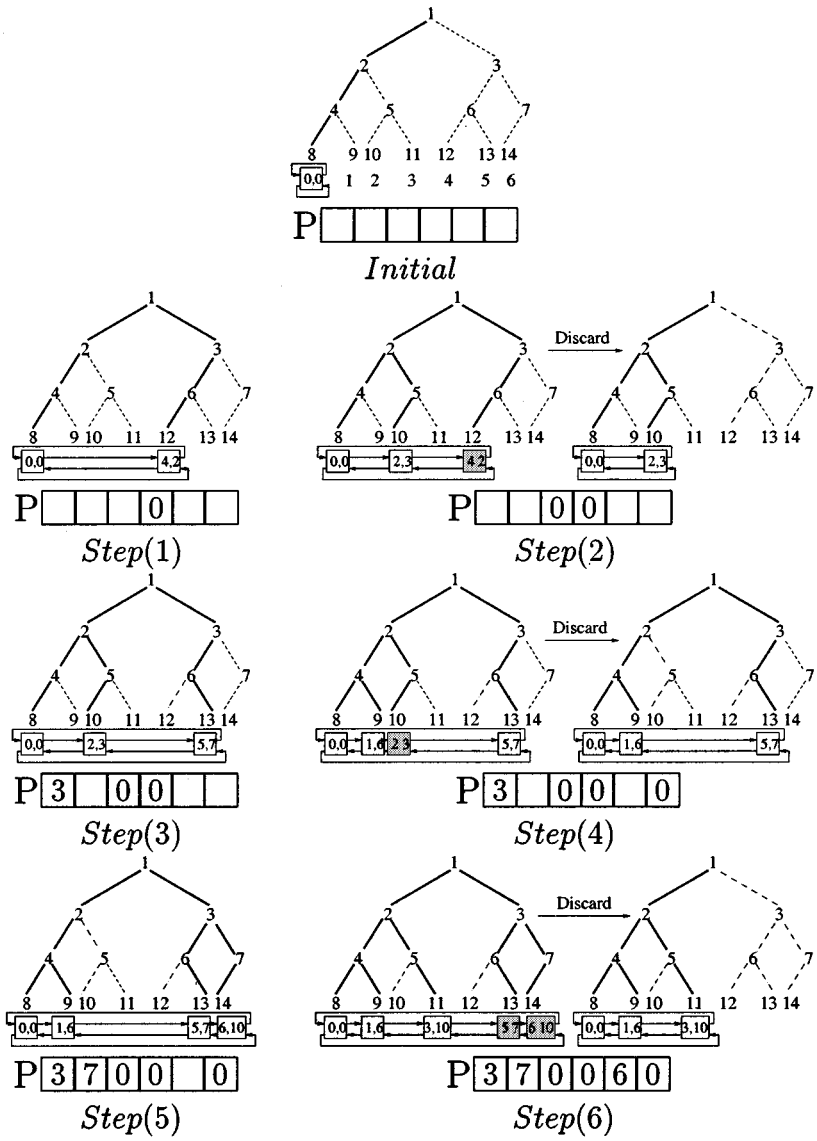


Fig. 6. Example of Algorithm 2 to compute the LCS of sequence pair ($\langle 4\ 3\ 1\ 6\ 2\ 5\rangle$, $\langle 6\ 3\ 5\ 4\ 1\ 2\rangle$). Dimensions of the six blocks are shown in Fig. 1. Here, $w(i)$ is the width of block i and P is the position of blocks.

TABLE I
TIME COMPARISON FOR SINGLE COMPUTATION BETWEEN ORIGINAL ALGORITHM AND OUR ALGORITHMS

block	original(s)	alg 1(s)	alg 2(s)
16	5.50e-05	3.39e-06	8.48e-06
32	1.92e-04	7.21e-06	1.74e-05
64	7.25e-04	1.63e-05	3.33e-05
128	2.83e-03	4.47e-05	6.96e-05
256	1.13e-02	1.13e-04	1.46e-04
512	5.68e-02	3.29e-04	3.09e-04
1024	0.249	8.87e-04	6.52e-04
2048	1.06	2.45e-03	1.53e-03
4096	4.37	9.29e-03	3.50e-03
8192	19.6	2.93e-02	8.20e-03
16384	121	9.47e-02	2.54e-02

TABLE II
TIME COMPARISON FOR PLACEMENT EXPERIMENTS BETWEEN THE ORIGINAL AND OURS

data	block	net	original(s)	ours (s)	speedup
apte	9	97	88	34	3X
xerox	10	203	164	64	2.5X
hp	11	83	86	30	3X
ami33	33	123	897	182	5X
ami49	49	408	1956	503	4X

logical routing). In the general case, the two sequences may have different length or repeated elements. For example, $X = \langle abcbba\rangle$, $Y = \langle babdadb\rangle$. For the common situation, each $\mathrm{match}[i]$ does not contain one element, but a list of arbitrary number of elements. These lists $\mathrm{match}[i]$, $i = 1, 2, \ldots, n$ must be kept in decreasing order on j such that $X[i] = Y[j]$. For the above sequences, $X = \langle abcbba\rangle$, $Y = \langle babdadb\rangle$, the match will be

$$\mathrm{match}[1] = \{5, 2\}$$
$$\mathrm{match}[2] = \{7, 3, 1\}$$

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF RUNTIME AND AREA AMONG O TREE (IN SUN ULTRA60), B* TREE (IN SUN ULTRA1), AND OURS (IN SUN ULTRA1)

circuit	block	O tree[11]		B* tree[1]		ours	
		time(Ultra60) (s)	area (mm^2)	time(Ultra1) (s)	area (mm^2)	time(Ultra1) (s)	area (mm^2)
apte	9	11	46.92	7	46.92	1	46.92
xerox	10	38	20.21	25	19.83	14	19.80
hp	11	19	9.159	55	8.95	6	8.947
ami33	33	119	1.242	3417	1.27	20	1.205
ami49	49	406	37.73	4752	36.80	31	36.50

$$\mathrm{match}[3] = \{\}$$
$$\mathrm{match}[4] = \mathrm{match}[2]$$
$$\mathrm{match}[5] = \mathrm{match}[2]$$
$$\mathrm{match}[6] = \mathrm{match}[1].$$

The array match can be created by sorting each sequence while keeping track of each element's original position. Then, we can merge the sorted sequences to construct match. This step can be done in $O(n)$ time on a fixed-size alphabet by using counting sort and in $O(n \log n)$ time on infinite alphabet by using heap sort or other sorting algorithms. Then, the complexity of our algorithms depends on the size of match. Let r denote the size of match, i.e., the number of elements in the set $\{(i, j) | X[i] = Y[j]\}$. The complexity of Algorithm 1 is $O(r \cdot n + n \log n)$ on an infinite alphabet, and $O(r \cdot n)$ on a fixed-size alphabet. The complexity of Algorithm 2 is $O(r \log\log n + n \log n)$ on an infinite alphabet and $O((n+r) \log\log n)$ on a fixed-size alphabet. In the worst case, r is $O(n^2)$, but in typical applications, where elements are repeated not more than a constant time, r is $O(n)$.

V. EXPERIMENTAL RESULTS

In this section, we first design experiments to compare our algorithms with the original graph-based algorithm in two aspects, evaluating a single sequence pair and obtaining block placement. Then, we show the comparison of placement results between our algorithm and others (including those based on O tree and B* tree).

We considered test cases of sequence pairs with 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, and 16 384 blocks. Table I lists the average runtime (in seconds) to evaluate a single sequence pair for the original algorithm and our algorithms. The original algorithm uses an efficient method to construct the constraint graphs in $O(n^2)$ time and uses an algorithm similar to that in [6] to find the longest path length to all vertices from source vertex in $O(n + m)$ time, where $n = \#\mathrm{blocks(vertices)}$, $m = \#\mathrm{edges}$. The experiment shows that both Algorithm 1 and Algorithm 2 are significantly faster than the original algorithm. For example, we achieved 60× speedup over the original algorithm when the number of blocks is 128. We can easily evaluate a million sequence pairs within one minute for typical input size of placement problems.

The comparison in block placement was carried out for the MCNC benchmarks. The cost function C we used to evaluate a sequence pair is defined as

$$C = A + \lambda \cdot W$$

where A is the area of the placement and W is the total wire length of all nets based on the half perimeter estimate of bounding box for each net where terminals are assumed to be at the center of each block. The weight parameter λ is used in the cost function for balancing the two factors. Note that, since we are only replacing the original sequence pair evaluation algorithm with Algorithm 1, the number of iterations and the final solutions are identical. Table II lists the comparison results.

Notably, the wiring cost estimations for the original and our method are the same. It accounts for a nontrivial part in the runtime. In spite of this, as we can see, our method still outperforms the original one and achieved 3.5× speedup on average. The larger the circuit, the greater speedup is gained.

We also compared our algorithm with block-placement algorithms based on O tree [2], [11] and B* tree [1] on MCNC benchmark problems. (Note that O tree and B* tree have reported the best results for these benchmark problems.) Table III lists the runtime and chip area for O tree, B* tree and ours. For comparison purpose, all experiments were optimizing chip area only, since only area minimization results were reported for B* tree in [1]. The runtime and areas for O tree and B* tree are directly taken from [11] and [1], respectively. The results of our algorithm are based on Sun SparcUltra 1 (166 MHz) with 128-MB memory, while B* tree is on Sun SparcUltra 1 (200 MHz) with 256-MB memory and O tree is on Sun Ultra 60, which is significantly faster. For all MCNC benchmark problems, ours has obtained the best results ever reported in the literature with significantly less runtime. For example, the best known result for ami49 (36.8 mm^2) was obtained by a B*-tree-based algorithm using 4752 s and we obtained a better result (36.5 mm^2) in 31 s. Upon using longer runtime, we can still get a little better results for ami33 and ami49. For example, 36.46 mm^2 was obtained for ami49 in 77 s and 1.195 mm^2 for ami33 in 74 s.

VI. CONCLUDING REMARKS

Sequence pair is an elegant topology representation of block placement. In this paper, we have proposed a new algorithm to translate a sequence pair to a block placement in $O(n \log\log n)$ time. Given a sequence pair, the algorithm not only evaluates the area of the placement, but also gives the position of each block, which is essential to wire-length estimation in practical VLSI placement. Experimental results show that the algorithm is indeed very fast so that we can use it to evaluate a million sequence pairs within one minute for typical input size of block-placement problems. For all MCNC benchmark block-place-

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.

ment problems, we have obtained the best results ever reported in the literature with significantly less runtime.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their helpful comments. In particular, they would like to thank one of the reviewers for pointing out that the algorithm in [15] can be modified to output all block positions.

REFERENCES

[1] Y. C. Chang, Y. W. Chang, G. M. Wu, and S. W. Wu, "B*-trees: A new representation for nonslicing floorplans," in *Proc. Design Automation Conf.*, June 2000, pp. 458–463.

[2] P. N. Guo, C. K. Cheng, and T. Yoshimura, "An O-tree representation of nonslicing floorplans and its applications," in *Proc. Design Automation Conf.*, June 1999, pp. 268–273.

[3] J. W. Hunt and T. G. Szymanski, "A fast algorithm for computing longest common subsequences," *Commun. ACM*, vol. 20, no. 5, pp. 350–353, May 1977.

[4] D. B. Johnson, "A priority queue in which initialization and queue operations take $O(\log \log D)$ time," *Math. Syst. Theory*, vol. 15, no. 4, pp. 295–309, Dec. 1982.

[5] M. Kang and W. W. M. Dai, "Topology constrained rectilinear block packing for layout reuse," in *Proc. Int. Symp. Physical Design*, Apr. 1998, pp. 179–186.

[6] J. A. McHugh, *Algorithmic Graph Theory*. Englewood Cliffs, NJ: Prentice-Hall, 1990.

[7] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "VLSI module placement based on rectangle-packing by the sequence pair," *IEEE Trans. Computer-Aided Design*, vol. 15, pp. 1518–1524, Dec. 1996.

[8] H. Murata, K. Fujiyoshi, and M. Kaneko, "VLSI/PCB placement with obstacles based on sequence pair," in *Proc. Int. Symp. Physical Design*, Apr. 1997, pp. 26–31.

[9] H. Murata and E. S. Kuh, "Sequence-pair based placement method for hard/soft/pre-placed modules," in *Proc. Int. Symp. Physical Design*, Apr. 1998, pp. 167–172.

[10] S. Nakatake, H. Murata, K. Fujiyoshi, and Y. Kajitani, "Module placement on BSG-structure and IC layout applications," in *Proc. Int. Conf. Computer-Aided Design*, Nov. 1996, pp. 484–491.

[11] Y. Pang, C. K. Cheng, and T. Yoshimura, "An enhanced perturbing algorithm for floorplan design using the O-tree representation," in *Proc. Int. Symp. Physical Design*, Apr. 2000, pp. 168–173.

[12] M. Sarrafzadeh and C. K. Wong, *An Introduction to VLSI Physical Design*. New York: McGraw-Hill, 1996.

[13] *National Technology Roadmap for Semiconductors*, Semiconductor Industry Association, San Jose, CA, 1997.

[14] N. Sherwani, *Algorithms for VLSI Physical Design Automation*. Norwell, MA: Kluwer, 1995.

[15] T. Takahashi, "An algorithm for finding a maximum-weight decreasing sequence in a permutation, motivated by rectangle packing problem," in *IEICE Tech. Rep. VLSI Design Technol.*, vol. VLD96, July 1996, pp. 31–35.

[16] P. van Emde Boas, R. Kaas, and Z. Zijlstra, "Design and implementation of an efficient priority queue," *Math. Syst. Theory*, vol. 10, no. 2, pp. 99–127, Apr. 1977.

[17] J. Xu, P. N. Guo, and C. K. Cheng, "Rectilinear block placement using sequence pair," in *Proc. Int. Symp. Physical Design*, Apr. 1998, pp. 173–178.

Xiaoping Tang received the B.S. degree in computer science from the University of Science and Technology of China, Hefei, in 1995 and the M.S. degree in computer science from the Institute of Software, Chinese Academy of Sciences, Beijing, in 1998. He is currently working toward the Ph.D. degree in computer sciences at the University of Texas at Austin, Austin.

His research interests include algorithms and computer-aided design in VLSI, primarily in the area of physical design. He is currently working on design planning, including floorplanning, buffer planning, and wire planning.

Ruiqi Tian received the B.S. degree in computer science in 1996 and the B.S. and M.A. degrees in physics in 1994 and 1997, respectively, from the University of Texas at Austin, Austin. He is currently working toward the Ph.D. degree in computer sciences at the same university.

He is currently with Motorola Inc., Austin, TX. His current research interest is computer-aided design for VLSI, especially in the area of design for manufacturability.

D. F. Wong (M'88) received the B.Sc. degree in mathematics from the University of Toronto, Toronto, ON, Canada, and the M.S. degree in mathematics and the Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign.

He is currently a Professor of Computer Sciences with the University of Texas at Austin, Austin. He has authored or coauthored over 200 technical papers and the book *Simulated Annealing for VLSI Design* (Norwell, MA: Kluwer, 1988) in addition to two invited articles in the *Wiley Encyclopedia of Electrical and Electronics Engineering* (1999). He has supervised 22 Ph.D. dissertations. He is on the Editorial Boards of *ACM Transactions on Design Automation of Electronic Systems* and *International Journal on Applied Mathematics.* He was the General Chair of the 1999 ACM International Symposium on Physical Design and was the Technical Program Chair of the same conference in 1998. He also has served on the technical program committees of many other VLSI conferences. His current research interest is computer-aided design of VLSI.

Dr. Wong received the 2000 IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYTEMS Best Paper Award for interconnect optimization and Best Paper Awards at the 1986 Design Automation Conference and the Internationl Conference on Computer Design for floorplan design and FPGA routing, respectively. He was an Associate Editor of IEEE TRANSACTIONS ON COMPUTERS and a Guest Editor of four special issues on physical design for IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYTEMS.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 08,2024 at 05:06:05 UTC from IEEE Xplore. Restrictions apply.